

Herança

1 - Fatorando Dados e Métodos Comuns na Superclasse e Herdando nas Subclasses

No contexto do conceito de herança, utilizado no Paradigma Orientado a Objetos (POO), é possível fatorar dados (atributos e referências) e métodos comuns a várias classes, denominadas subclasses, definindo-os em uma classe mais genérica, denominada superclasse. Desta forma, as subclasses herdam os dados e métodos comuns, e acrescentam dados e métodos específicos.

No nosso projeto de referência, a classe mais geral (superclasse) **Veículo**, armazena os dados e métodos comuns às classes mais específicas (subclasses) **Carro** e **Caminhão**: **marca_modelo**, **alimentação** (flex, diesel, elétrico), **potência** e **importado**. A classe **Carro**, herda os atributos comuns da classe **Veículo** e define adicionalmente atributos específicos: **tipo** (passeio, utilitário, caminhonete) e **volume_carga**. A classe **Caminhão**, herda os atributos comuns da classe **Veículo** e define adicionalmente atributos específicos: **tração** (6x4, 6x2, 4x4) e **capacidade_carga**.

A grande vantagem da utilização do conceito de herança, é que os dados e métodos comuns são herdados pelas subclasses. Se for necessário alterá-los, a alteração pode ser feita em um único lugar: no código da superclasse. No entanto, é preciso tomar cuidado na escolha da superclasse. Uma restrição importante para especificar uma superclasse, é mapear uma entidade do mundo real, que corresponda a um tipo mais genérico para todas as suas subclasses. Para avaliar se a especificação está adequada, é preciso responder a seguinte pergunta: "Uma subclasse é um tipo da superclasse?". Observe que podemos afirmar que **Carro** é um tipo de **Veículo**. De forma semelhante, podemos afirmar: **Caminhão** é um tipo de **Veículo**.

Mas você não pode definir uma superclasse simplesmente porque deseja herdar seus dados e métodos em outras classes, se isso não fizer sentido no mundo real. Suponha, por exemplo que você defina o atributo **potência** na classe **Motor** e você deseje utilizar esse atributo na classe **Carro**. Embora seja conveniente, você não pode dizer que **Carro** é um tipo de **Motor**. Portanto, ao invés de utilizar herança neste caso, você poderá optar por criar uma referência para um objeto da classe **Motor** na classe **Carro**.

A implementação da superclasse **Veículo** é a mesma ilustrada no Tutorial 1. Vamos então ilustrar as implementações das subclasses: **Carro** e **Caminhão**.

`class Carro(Veículo):`

```
def __init__(self, marca_modelo, alimentação, potência, importado, tipo, volume_carga):
    super().__init__(marca_modelo, alimentação, potência, importado)
    self.tipo = tipo if tipo in ('passeio', 'utilitário', 'caminhonete') else 'indefinida'
    self.volume_carga = volume_carga

def __str__(self):
    if self.importado: importado_str = 'importado '
    else: importado_str = ''
    formato = '{<19> } {<8> } {<6> } {<11> } {>8} } }'
    carro_formatado = formato.format('|', self.marca_modelo, '|', self.alimentação, '|', f'{self.potência:3d}' + ' cv',
                                     '|', self.tipo, '|', f'{self.volume_carga:4d}' + ' l', '|', importado_str)
    return carro_formatado
```

```
class Caminhão(Veículo):
```

```
    def __init__(self, marca_modelo, alimentação, potência, importado, tração, capacidade_carga):
        super().__init__(marca_modelo, alimentação, potência, importado)
        self.tração = tração if tração in ('6x4', '6x2', '4x4') else 'sem tração'
        self.capacidade_carga = capacidade_carga

    def __str__(self):
        if self.importado: importado_str = 'importado '
        else: importado_str = ''
        formato = '{: <19} {} {: <8} {} {: <6} {} {: <11} {} {: <6} {} {}'
        caminhão_formatado = formato.format(' ', self.marca_modelo, ' ', self.alimentação, ' ', f'{self.potência:3d}' + ' cv',
                                             ' ', self.tração, ' ', f'{self.capacidade_carga:5d}' + ' kg', ' ', importado_str)
        return caminhão_formatado
```

Na definição das subclasses existem diferenças, que serão ilustradas tomando como base a subclasse **Carro**:

- definição da superclasse importada

```
class Carro(Veículo):
```
- o método `__init__` recebe adicionalmente como argumentos os dados herdados da superclasse

```
def __init__(self, marca_modelo, alimentação, potência, importado, tipo,
             volume_carga):
```
- na primeiro comando do método `__init__`, os parâmetros que receberam os dados herdados são repassados para o método `__init__` da superclasse

```
super().__init__(marca_modelo, alimentação, potência, importado)
```

Observe que os métodos das subclasses acessam diretamente os dados (atributos ou referências) herdados da superclasse: **marca_modelo**, **alimentação**, **potência** e **importado**.

Neste ponto é importante esclarecer o conceito de sobreposição de métodos. Observe que um método herdado da superclasse, pode ter o seu bloco de implementação redefinido, conservando sua assinatura (nome e lista de parâmetros) como ocorre, por exemplo, com o método `__str__` definido na superclasse e sobreposto nas subclasses. Se o método `__str__` for chamado a partir de um objeto da subclasse, será utilizada a implementação sobreposta do método `__str__`. Obviamente que se um método herdado não for sobreposto, ele passa a ser a única opção para ser chamado na subclasse, pois não vão existir métodos para substituí-los nas subclasses.

2 - Atualizando a Função de Filtragem de Objetos da Classe de Associação

Para incorporar filtros para as subclasses **Carro** e **Caminhão**, a implementação do método **selecionar_lançamentos** do módulo **lançamento** foi atualizada com dois filtros adicionais: **tipo_carro** e **capacidade_mínima_carga_caminhão**, conforme ilustrado na próxima página.

Antes de testar o filtro relacionado com uma dada subclasse, é necessário testar se o objeto inicialmente armazenado em uma variável denotando a superclasse (**lançamento.veículo**) pertence à subclasse desejada. Por exemplo, para testar se o objeto armazenado na variável **lançamento.veículo** é um objeto da subclasse **Carro**, é implementado o seguinte teste:

```
if isinstance(lançamento.veículo, Carro):
```

De resto, a implementação da função **selecionar_lançamentos**, ilustrada no Tutorial 2, se mantém inalterada.

```
def selecionar_lançamentos(data_mínima_lançamento=None, potência_mínima_veículo=None, tipo_carro=None,
                           capacidade_mínima_carga_caminhão=None, ranking_máximo_avaliação_agência_publicidade=None,
                           país_origem_montadora=None):
    filtros = "\nFiltros --"
    if data_mínima_lançamento is not None: filtros += 'data mínima de lançamento: ' + str(data_mínima_lançamento)
    if potência_mínima_veículo is not None: filtros += ' - potência mínima do veículo: ' + str(potência_mínima_veículo)
    if tipo_carro is not None: filtros += ('\n - tipo do carro: ' + tipo_carro)
    if capacidade_mínima_carga_caminhão is not None:
        filtros += (' - capacidade mínima de carga do caminhão: ' + str(capacidade_mínima_carga_caminhão))
    if ranking_máximo_avaliação_agência_publicidade is not None:
        filtros += ('\n - ranking máximo de avaliação da agência de publicidade: ' + str(ranking_máximo_avaliação_agência_publicidade))
    if país_origem_montadora is not None: filtros += (' - país de origem da montadora: ' + país_origem_montadora)
    lançamentos_selecionados = []
    for lançamento in lançamentos:
        if data_mínima_lançamento is not None and lançamento.data < data_mínima_lançamento: continue
        excluir_lançamento = False
        for veículo in lançamento.montadora.veículos.values():
            if potência_mínima_veículo is not None and veículo.potência < potência_mínima_veículo:
                excluir_lançamento = True
                break
            if isinstance(veículo, Carro):
                if tipo_carro is not None and veículo.tipo != tipo_carro:
                    excluir_lançamento = True
                    break
            elif isinstance(veículo, Caminhão):
                if capacidade_mínima_carga_caminhão is not None and veículo.capacidade_carga < capacidade_mínima_carga_caminhão:
                    excluir_lançamento = True
                    break
        if excluir_lançamento: continue
        if (ranking_máximo_avaliação_agência_publicidade is not None
            and lançamento.agência_publicidade.ranking_avaliação > ranking_máximo_avaliação_agência_publicidade): continue
        if país_origem_montadora is not None and lançamento.montadora.país_origem != país_origem_montadora: continue
        lançamentos_selecionados.append(lançamento)
    return filtros, lançamentos_selecionados
```

Adicionalmente, na classe `Lançamento` é necessário testar a subclasse no método `str_atributos_veículos` e atualizar o posicionamento das colunas no método `str_filtro`.

```
def str_atributos_veículos(self):
    atributos_veículos_str = ""
    for índice, veículo in enumerate(self.montadora.veículos.values()):
        if (índice > 0): atributos_veículos_str += ' -- '
        atributos_veículos_str += f'{veículo.potência:3d}' + ' cv - '
        if isinstance(veículo, Carro): atributos_veículos_str += veículo.tipo
        elif isinstance(veículo, Caminhão): atributos_veículos_str += f'{veículo.capacidade_carga:5d}' + ' kg'
    return atributos_veículos_str

def str_filtro(self):
    formato = '{:>2} {} {:<14} {} {:<44} {}'
    filtro_formatado = formato.format(str(self.agência_publicidade.ranking_avaliação), '|',
                                      self.montadora.país_origem, '|', self.str_atributos_veículos(), '|')
    return self.__str__() + filtro_formatado
```

3 - Atualizando a Implementação do Módulo do Diretório de Controle

No módulo `projeto` do diretório `controle`, foram realizadas duas atualizações. A primeira alteração ocorreu na função `cadastrar_montadoras`: passaram a ser associados a cada objeto da classe `Montadora` objetos das subclasses `Carro` e `Caminhão`, ao invés de objetos da classe `Veículo`.

```
def cadastrar_montadoras():
    montadora = Montadora(nome='Hyundai', pais_origem='Coréia do Sul', sede_maior_fabrica_brasil='Piracicaba - SP')
    inserir_montadora(montadora)
    montadora.inserir_veiculo(Carro(marca_modelo='Hyundai HB20', alimentacao='flex', potencia=80, importado=False, tipo='passeio',
                                     volume_carga=475))
    montadora = Montadora('GM', 'Estados Unidos', 'São Caetano - SP')
    inserir_montadora(montadora)
    montadora.inserir_veiculo(Carro('Chevrolet Onix Plus', 'flex', 116, False, 'passeio', 500))
    montadora = Montadora('Fiat', 'Itália', 'Betim - MG')
    inserir_montadora(montadora)
    montadora.inserir_veiculo(Carro('Fiat Strada', 'diesel', 130, False, 'caminhonete', 1354))
    montadora.inserir_veiculo(Carro('Fiat Toro', 'diesel', 185, False, 'caminhonete', 1225))
    montadora = Montadora('BYD', 'China', 'Camaçari - BA')
    inserir_montadora(montadora)
    montadora.inserir_veiculo(Carro('BYD Dolphin', 'elétrico', 95, True, 'passeio', 250))
    montadora = Montadora('Volvo', 'Suécia', 'exterior')
    inserir_montadora(montadora)
    montadora.inserir_veiculo(Carro('Volvo XC40', 'elétrico', 231, True, 'passeio', 419))
    montadora = Montadora('Volvo Caminhões', 'Suécia', 'Curitiba - PR')
    inserir_montadora(montadora)
    montadora.inserir_veiculo(Caminhão(marca_modelo='Volvo FH 540', alimentacao='diesel', potencia=540, importado=False,
                                       tração='6x4', capacidade_carga=80000))
    montadora.inserir_veiculo(Caminhão('Volvo FH 460', 'diesel', 460, False, '6x2',
                                       38000))
    montadora = Montadora('Volkswagen Caminhões', 'Alemanha', 'Resende - RJ')
    inserir_montadora(montadora)
    montadora.inserir_veiculo(Caminhão('VW Delivery 11180', 'diesel', 175, False, '4x4', 7500))
    montadora = Montadora('DAF', 'Holanda', 'Ponta Grossa - PR')
    inserir_montadora(montadora)
    montadora.inserir_veiculo(Caminhão('DAF XF 530', 'diesel', 530, True, '6x4', 74000))
```

No corpo principal do projeto alterações para incluir os atributos e filtros das subclasses.

```
if __name__ == '__main__':
    cadastrar_agencias_publicidade()
    imprimir_objetos(cabeçalho='\nAgências de Publicidades : nome - país de origem - ranking de avaliação',
                    objetos=get_agencias_publicidade().values())
    cadastrar_montadoras()
    imprimir_objetos(cabeçalho='\nMontadoras : nome - país de origem - sede da maior fábrica no Brasil',
                    objetos=get_montadoras().values())
    print('\nMontadoras : nome - país de origem - sede da maior fábrica no Brasil'
          + '\n - Veículos : marca modelo - alimentação - potência - carro:[tipo - volume_carga] | caminhão:[tração - capacidade de carga]'
          + ' - importado')
    for indice, montadora in enumerate(get_montadoras().values()):
        imprimir_objeto(indice=indice, objeto_str=str(montadora))
        imprimir_objetos_internos(montadora.veiculos.values())
    cadastrar_lancamentos()
    cabeçalho_lancamento = ('Lançamento : nome da montadora - marca modelo do veículo - nome da agência de publicidade'
                             + ' - data do lançamento')
    imprimir_objetos('\n' + cabeçalho_lancamento, get_lancamentos())
    filtros, lancamentos_selecionados = selecionar_lancamentos()
    cabeçalho_lancamento_filtros = (cabeçalho_lancamento
                                     + '\n -- ranking de avaliação da agência de publicidade - país de origem da montadora'
                                     + '\n - veículos [ potência do veículo - tipo do carro | capacidade de carga do caminhão ] ')
    imprimir_objetos_associacao_filtros(cabeçalho_lancamento_filtros, lancamentos_selecionados, filtros)
    filtros, lancamentos_selecionados = selecionar_lancamentos(data_mínima_lancamento=Data(dia=1, mês=1, ano=2012))
    imprimir_objetos_associacao_filtros(cabeçalho_lancamento_filtros, lancamentos_selecionados, filtros)
    filtros, lancamentos_selecionados = selecionar_lancamentos(Data(1, 1, 2012), potencia_mínima_veiculo=90)
    imprimir_objetos_associacao_filtros(cabeçalho_lancamento_filtros, lancamentos_selecionados, filtros)
    filtros, lancamentos_selecionados = selecionar_lancamentos(Data(1, 1, 2012), 90, tipo_carro='passeio')
    imprimir_objetos_associacao_filtros(cabeçalho_lancamento_filtros, lancamentos_selecionados, filtros)
    filtros, lancamentos_selecionados = selecionar_lancamentos(Data(1, 1, 2012), 90, 'passeio',
                                                                capacidade_mínima_carga_caminhão=20000)
    imprimir_objetos_associacao_filtros(cabeçalho_lancamento_filtros, lancamentos_selecionados, filtros)
    filtros, lancamentos_selecionados = selecionar_lancamentos(Data(1, 1, 2012), 90, 'passeio', 20000,
                                                                ranking_máximo_avaliação_agência_publicidade=3, )
    imprimir_objetos_associacao_filtros(cabeçalho_lancamento_filtros, lancamentos_selecionados, filtros)
    filtros, lancamentos_selecionados = selecionar_lancamentos(Data(1, 1, 2012), 90, 'passeio', 20000, 3,
                                                                pais_origem_montadora='Suécia')
    imprimir_objetos_associacao_filtros(cabeçalho_lancamento_filtros, lancamentos_selecionados, filtros)
```

A sobreposição de métodos é conhecida na Programação Orientada a Objetos como polimorfismo. Imagine que o maestro de uma orquestra está regendo a interpretação de uma determinada música. Cada instrumentista reage aos gestos de regência do maestro de acordo com o instrumento que está tocando. De forma análoga, você pode definir um método na superclasse com uma dada assinatura (nome e lista de parâmetros) e sobrepor a implementação desses métodos nas subclasses (mantendo a assinatura, mas alterando a implementação).

Observe que a Etapa 3 do projeto de referência ilustra bem o conceito de polimorfismo. Você cadastra objetos das subclasses no dicionário `veículos`. Quando o método `imprimir_objetos` é executado, os métodos sobrepostos `__str__` das subclasses são priorizados em relação ao método herdado `__str__` da superclasse. Este é exatamente o comportamento esperado, pois somente os métodos sobrepostos `__str__` concatenam os dados que são definidos somente nas subclasses, e quando você quer imprimir os dados de um carro, você não quer ver somente os dados herdados da superclasse `Veículo` (`marca_modelo`, `alimentação`, `potência` e `importado`), mas também os dados específicos da subclasse `Carro` (`tipo` e `volume_carga`).

Para ilustrar a execução da Etapa 3 do projeto de referência serão mostradas somente as alterações: (a) impressão das montadoras e seus respectivos veículos; e (b) filtragem dos lançamentos.

A saída com a impressão das montadoras e seus respectivos veículos é ilustrada a seguir.

```
Montadoras : nome - país de origem - sede da maior fábrica no Brasil
- Veículos : marca modelo - alimentação - potência - carro:[tipo - volume_carga] | caminhão:[tração - capacidade de carga] - importado
1 - | Hyundai | Coréia do Sul | Piracicaba - SP |
- | Hyundai HB20 | flex | 80 cv | passeio | 475 l |
2 - | GM | Estados Unidos | São Caetano - SP |
- | Chevrolet Onix Plus | flex | 116 cv | passeio | 500 l |
3 - | Fiat | Itália | Betim - MG |
- | Fiat Strada | diesel | 130 cv | caminhonete | 1354 l |
- | Fiat Toro | diesel | 185 cv | caminhonete | 1225 l |
4 - | BYD | China | Camaçari - BA |
- | BYD Dolphin | elétrico | 95 cv | passeio | 250 l | importado |
5 - | Volvo | Suécia | exterior |
- | Volvo XC40 | elétrico | 231 cv | passeio | 419 l | importado |
6 - | Volvo Caminhões | Suécia | Curitiba - PR |
- | Volvo FH 540 | diesel | 540 cv | 6x4 | 80000 kg |
- | Volvo FH 460 | diesel | 460 cv | 6x2 | 38000 kg |
7 - | Volkswagen Caminhões | Alemanha | Resende - RJ |
- | VW Delivery 11180 | diesel | 175 cv | 4x4 | 7500 kg |
8 - | DAF | Holanda | Ponta Grossa - PR |
- | DAF XF 530 | diesel | 530 cv | 6x4 | 74000 kg | importado |
```

A saída com a filtragem dos lançamentos é ilustrada a seguir.

Filtros --

Lançamento : nome da montadora - marca modelo do veículo - nome da agência de publicidade - data do lançamento

-- ranking de avaliação da agência de publicidade - país de origem da montadora

- veículos [potência do veículo - tipo do carro | capacidade de carga do caminhão]

1 -	Hyundai	Africa	01/09/2012	5	Coréia do Sul	80 cv - passeio	
2 -	GM	McCann	12/09/2019	4	Estados Unidos	116 cv - passeio	
3 -	Fiat	Galeria	01/09/1998	2	Itália	130 cv - caminhonete -- 185 cv - caminhonete	
4 -	Fiat	Galeria	01/09/2016	2	Itália	130 cv - caminhonete -- 185 cv - caminhonete	
5 -	BYD	Artplan	28/06/2023	3	China	95 cv - passeio	
6 -	Volvo	Artplan	15/04/2018	3	Suécia	231 cv - passeio	
7 -	Volvo Caminhões	BETC Havas	01/03/2014	1	Suécia	540 cv - 80000 kg -- 460 cv - 38000 kg	
8 -	Volkswagen Caminhões	Galeria	23/01/2018	2	Alemanha	175 cv - 7500 kg	
9 -	DAF	Galeria	18/08/2020	2	Holanda	530 cv - 74000 kg	
10 -	Volvo Caminhões	BETC Havas	17/04/2014	1	Suécia	540 cv - 80000 kg -- 460 cv - 38000 kg	

Filtros -- data mínima de lançamento: 01/01/2012

Lançamento : nome da montadora - marca modelo do veículo - nome da agência de publicidade - data do lançamento

-- ranking de avaliação da agência de publicidade - país de origem da montadora

- veículos [potência do veículo - tipo do carro | capacidade de carga do caminhão]

1 -	Hyundai	Africa	01/09/2012	5	Coréia do Sul	80 cv - passeio	
2 -	GM	McCann	12/09/2019	4	Estados Unidos	116 cv - passeio	
3 -	Fiat	Galeria	01/09/2016	2	Itália	130 cv - caminhonete -- 185 cv - caminhonete	
4 -	BYD	Artplan	28/06/2023	3	China	95 cv - passeio	
5 -	Volvo	Artplan	15/04/2018	3	Suécia	231 cv - passeio	
6 -	Volvo Caminhões	BETC Havas	01/03/2014	1	Suécia	540 cv - 80000 kg -- 460 cv - 38000 kg	
7 -	Volkswagen Caminhões	Galeria	23/01/2018	2	Alemanha	175 cv - 7500 kg	
8 -	DAF	Galeria	18/08/2020	2	Holanda	530 cv - 74000 kg	
9 -	Volvo Caminhões	BETC Havas	17/04/2014	1	Suécia	540 cv - 80000 kg -- 460 cv - 38000 kg	

Filtros -- data mínima de lançamento: 01/01/2012 - potência mínima do veículo: 90

Lançamento : nome da montadora - marca modelo do veículo - nome da agência de publicidade - data do lançamento

-- ranking de avaliação da agência de publicidade - país de origem da montadora

- veículos [potência do veículo - tipo do carro | capacidade de carga do caminhão]

1 -	GM	McCann	12/09/2019	4	Estados Unidos	116 cv - passeio	
2 -	Fiat	Galeria	01/09/2016	2	Itália	130 cv - caminhonete -- 185 cv - caminhonete	
3 -	BYD	Artplan	28/06/2023	3	China	95 cv - passeio	
4 -	Volvo	Artplan	15/04/2018	3	Suécia	231 cv - passeio	
5 -	Volvo Caminhões	BETC Havas	01/03/2014	1	Suécia	540 cv - 80000 kg -- 460 cv - 38000 kg	
6 -	Volkswagen Caminhões	Galeria	23/01/2018	2	Alemanha	175 cv - 7500 kg	
7 -	DAF	Galeria	18/08/2020	2	Holanda	530 cv - 74000 kg	
8 -	Volvo Caminhões	BETC Havas	17/04/2014	1	Suécia	540 cv - 80000 kg -- 460 cv - 38000 kg	

Filtros -- data mínima de lançamento: 01/01/2012 - potência mínima do veículo: 90

- tipo do carro: passeio

Lançamento : nome da montadora - marca modelo do veículo - nome da agência de publicidade - data do lançamento

-- ranking de avaliação da agência de publicidade - país de origem da montadora

- veículos [potência do veículo - tipo do carro | capacidade de carga do caminhão]

1 -	GM	McCann	12/09/2019	4	Estados Unidos	116 cv - passeio	
2 -	BYD	Artplan	28/06/2023	3	China	95 cv - passeio	
3 -	Volvo	Artplan	15/04/2018	3	Suécia	231 cv - passeio	
4 -	Volvo Caminhões	BETC Havas	01/03/2014	1	Suécia	540 cv - 80000 kg -- 460 cv - 38000 kg	
5 -	Volkswagen Caminhões	Galeria	23/01/2018	2	Alemanha	175 cv - 7500 kg	
6 -	DAF	Galeria	18/08/2020	2	Holanda	530 cv - 74000 kg	
7 -	Volvo Caminhões	BETC Havas	17/04/2014	1	Suécia	540 cv - 80000 kg -- 460 cv - 38000 kg	

Linguagem de Programação I - Tutorial 3 - 7/7

Filtros -- data mínima de lançamento: 01/01/2012 - potência mínima do veículo: 90

- tipo do carro: passeio - capacidade mínima de carga do caminhão: 20000

Lançamento : nome da montadora - marca modelo do veículo - nome da agência de publicidade - data do lançamento

-- ranking de avaliação da agência de publicidade - país de origem da montadora

- veículos [potência do veículo - tipo do carro | capacidade de carga do caminhão]

1 -		GM			McCann			12/09/2019		4		Estados Unidos		116 cv - passeio		
2 -		BYD			Artplan			28/06/2023		3		China			95 cv - passeio	
3 -		Volvo			Artplan			15/04/2018		3		Suécia			231 cv - passeio	
4 -		Volvo Caminhões			BETC Havas			01/03/2014		1		Suécia			540 cv - 80000 kg -- 460 cv - 38000 kg	
5 -		DAF			Galeria			18/08/2020		2		Holanda			530 cv - 74000 kg	
6 -		Volvo Caminhões			BETC Havas			17/04/2014		1		Suécia			540 cv - 80000 kg -- 460 cv - 38000 kg	

Filtros -- data mínima de lançamento: 01/01/2012 - potência mínima do veículo: 90

- tipo do carro: passeio - capacidade mínima de carga do caminhão: 20000

- ranking máximo de avaliação da agência de publicidade: 3

Lançamento : nome da montadora - marca modelo do veículo - nome da agência de publicidade - data do lançamento

-- ranking de avaliação da agência de publicidade - país de origem da montadora

- veículos [potência do veículo - tipo do carro | capacidade de carga do caminhão]

1 -		BYD			Artplan			28/06/2023		3		China			95 cv - passeio	
2 -		Volvo			Artplan			15/04/2018		3		Suécia			231 cv - passeio	
3 -		Volvo Caminhões			BETC Havas			01/03/2014		1		Suécia			540 cv - 80000 kg -- 460 cv - 38000 kg	
4 -		DAF			Galeria			18/08/2020		2		Holanda			530 cv - 74000 kg	
5 -		Volvo Caminhões			BETC Havas			17/04/2014		1		Suécia			540 cv - 80000 kg -- 460 cv - 38000 kg	

Filtros -- data mínima de lançamento: 01/01/2012 - potência mínima do veículo: 90

- tipo do carro: passeio - capacidade mínima de carga do caminhão: 20000

- ranking máximo de avaliação da agência de publicidade: 3 - país de origem da montadora: Suécia

Lançamento : nome da montadora - marca modelo do veículo - nome da agência de publicidade - data do lançamento

-- ranking de avaliação da agência de publicidade - país de origem da montadora

- veículos [potência do veículo - tipo do carro | capacidade de carga do caminhão]

1 -		Volvo			Artplan			15/04/2018		3		Suécia			231 cv - passeio	
2 -		Volvo Caminhões			BETC Havas			01/03/2014		1		Suécia			540 cv - 80000 kg -- 460 cv - 38000 kg	
3 -		Volvo Caminhões			BETC Havas			17/04/2014		1		Suécia			540 cv - 80000 kg -- 460 cv - 38000 kg	

Neste ponto, finalizamos o Tutorial 3 com a ilustração completa da Etapa 3 do projeto de referência.